

Audion Image Tools - User Guide

This is the English companion to `USER_GUIDE_RU.md`.

Purpose

Audion Image Tools is a portable GUI/CLI toolkit for batch image and PDF processing: conversion, PDF image extraction, image-to-PDF, crop, resize, DPI metadata, color profiles, printable sheets, contact sheets, and diagnostics.

Primary launch:

```
launcher_gui.cmd
```

Quick Route

1. Put files into `input\`, stage them from the GUI, or choose a folder in `SOURCE`.
2. Open `launcher_gui.cmd`.
3. Choose a ROOT module: `Conversion`, `PDF`, `Crop`, `Resize`, `Color`, `Photo sheet`, `Sheets and labels`, `Contact sheet`, or `Diagnostics`.
4. Use the action grid at the top of the module. Quick actions run immediately; parameterized actions show their settings in the same window.
5. Check `SOURCE`, `DESTINATION`, and the visible settings.
6. Click the amber `RUN` button when the action has parameters.
7. Read results from `output\` or from the selected `DESTINATION`.

Empty `SOURCE` and `DESTINATION` fields use command defaults, usually `input\` and a subfolder under `output\`.

New GUI Layout

The left navigation contains ROOT modules only. **Conversion** opens directly as a working window. Other modules show action buttons at the top and the selected action settings below.

Action buttons and **RUN** use a muted amber highlight with light gray text. Navigation buttons remain standard blue Quasar buttons. Settings are grouped into source, output/result, format, parameters, and options blocks.

Short labels are intentional. Longer explanations live in tooltips on headings, buttons, checkboxes, radio controls, and toggles. The tooltip background is **RGB(23, 33, 43)**. The **BACK** button has no tooltip.

Modules

Conversion

The Conversion window contains quick actions at the top and the normal conversion settings below.

Quick actions:

- **JPG 75% quality**
- **PNG**
- **Grayscale PNG**
- **2160p JPG 90% quality**
- **16:9 JPG 90% quality**

Convert images supports **JPG**, **PNG**, **TIFF**, **WebP**, **AVIF**, **HEIF**, and **HEIC**, with 60/75/90 presets and an exact 1..100 quality field defaulting to 83. DPI writing is available for **JPG**, **PNG**, and **TIFF**.

The **2160p JPG 90% quality** preset uses Lanczos. When the source is actually upscaled to UHD, a mild **Unsharp Mask** is applied after resizing. Downscale to UHD does not add extra sharpening.

PDF

Actions:

- **PDF export** - export pages by embedded images or full render.
- **Extract from PDF** - extract embedded raster objects, defaulting to original format.

- **Images to PDF** - build PDF files from images.

PDF export defaults to embedded raster extraction when possible. If the full page appearance is needed, choose render mode and set DPI. Output format and quality are controlled in the **Format** block.

PDF rasterize renders full PDF pages into images at 150/300/600 DPI. Use it when the PDF page itself matters, not only the embedded raster objects inside it.

Extract from PDF defaults to writing embedded images in their original format. You can also convert them to target formats, choose flat output or per-PDF folders, and optionally write TIFF sidecars next to PNG.

Images to PDF defaults to separate PDFs by folder, preserved source DPI, and lossless PNG PDF mode. A custom PDF DPI becomes active only when DPI preservation is off.

Crop

Actions:

- **Trim border** - remove a solid frame using the corner color.
- **Smart white crop** - trim white background to meaningful pixels.
- **Split...** - split images into strips or rectangular grid cells.

Safety margin, mm = 0 cuts exactly to the detected content. A value above zero keeps source background around the result using image DPI; if DPI is missing, 300 DPI is used.

Trim border and **Smart white crop** default to PNG, but output can be changed to **JPG**, **TIFF**, **WebP**, **AVIF**, **HEIF**, or **HEIC**. Crop actions can also build a PDF after cropping with embedded PNG or embedded JPG 75% parts without changing pixel resolution.

Split... writes parts as **name_01.png**, **name_02.png**, and so on. **Strips** uses direction and part count; **Grid** uses row and column counts.

Resize

Actions:

- **Screen** - 1080p, 1440p, or 2160p/UHD.
- **Aspect** - 16:9, A4, or A3.
- **Downscale** - 25%, 50%, a custom percent, or automatic fit to a target file size in MB.
- **Roll** - write DPI for a target physical side without changing pixels.

- **DPI** - write DPI only.

Screen -> 2160p applies a mild Unsharp Mask only on real upscale. **Downscale** applies a mild Unsharp Mask after Lanczos by default. Scale modes are **25%**, **50%**, **Custom %**, and **Fit**. **Fit** searches for the largest percent that keeps each output file below the target MB value; the target spinner steps by **0.1 MB**, and **0.8-1.0** of the target is treated as a good hit.

Resize algorithms:

- **Lanczos** - best visual quality for photos and soft images, but it can create halos and inflate PNG maps.
- **Bicubic** - good photo/JPG compromise: smoother than simple averaging, but it can add intermediate colors around sharp map lines.
- **Box** - dry averaging filter for maps, plans, screenshots, and flat graphics; usually fewer halos and smaller PNG than **Lanczos** / **Bicubic**.
- **Nearest** - creates no new colors and is often the smallest PNG option, but diagonals, labels, and thin lines can become jagged.

Practical rule: start with **Lanczos** or **Bicubic** for photos/JPG; for PNG maps and plans, use **Nearest** for minimum size or **Box** for a calmer visual compromise.

Tooltip details:

- **Fit** does not change JPG quality by itself. It searches for a resize percent that fits the target file size. For JPG, the final size depends on scale, quality, and image complexity; for PNG, it depends on color count, sharp edges, and the resize algorithm.
- **Target MB** is a per-file limit. If the target is **1.0**, the engine tries to land close to 1 MB, but exact byte-level matching is not expected; about **0.8-1.0** of the target is treated as a good result.
- **Lanczos** is attractive for photos and soft scans, but on maps it can create semi-transparent transitions around hard lines. That often increases PNG size.
- **Bicubic** is useful when you want a softer result than **Box** / **Nearest** without the full cost and halo tendency of **Lanczos**.
- **Box** is usually the safest first try for maps, plans, diagrams, and screenshots: it averages pixels simply and tends to create fewer halos.
- **Nearest** is useful when original colors and minimum PNG size matter, but it does not smooth diagonals or small text.

Color

Normalize to keep, sRGB, or CMYK, then choose output formats, quality, and metadata policy.

Photo Sheet

Build printable sheets. **Paper saver** creates dense rows without gaps. **Cut-lines** groups similar heights and writes **cut_plan.txt**. By default, source pixels are not resampled: sheet width and DPI describe the final output sheet. If the selected DPI is too low for the unchanged source pixels at the selected sheet width, the engine raises the sheet DPI automatically and reports a warning in the GUI/report.

Sheets And Labels

Tile sheet repeats one image on A5/A4/A3 or a roll. **Split TIFF** exports multi-page TIFF frames to PNG. **Watermark** adds either a horizontal corner label or a diagonal protective text mark.

Watermark has two modes. **Corner** places normal horizontal text into the selected corner with a controlled margin and font size. **Diagonal protective** centers text across the image diagonal and auto-fits the font size to the selected diagonal coverage; **60%** is a practical default for protective document marking and leaves room for short large words such as **COPY**. The font is intentionally simple and readable: Arial Bold when available. Opacity is adjustable separately: **0** is invisible, **255** is fully opaque. Default **64** is about 25% opacity and is chosen for print, where watermarks usually look darker than on screen.

Watermark tooltip details:

- **Corner** is for tidy technical marking: label, date, author, or draft status.
- **Diagonal protective** is for discouraging misuse: the text crosses the document and remains visible even after partial cropping.
- **Diagonal coverage, %** controls target text length relative to the image diagonal, not the font size directly. Shorter text becomes larger at the same coverage percent.
- **Corner font size** applies only to **Corner**. In diagonal mode the font size is calculated automatically.
- **Opacity** is the **0..255** alpha channel. Default **64** is about 25% opacity. For red or black diagonal text, start around **64-96**; for light text on dark images, start around **96-128**.
- **Color** in the GUI is selected from human presets with visible swatches: **Red**, **Black**, **White**, **Yellow**, **Gray**, **Blue**. For exact colors, use **Custom RGB** with **R**, **G**, and **B** channels from **0** to **255**. CLI users can still pass an HTML color or RGB manually, for example **#cc0000** or **204,0,0**.

Contact Sheet

Create an inspection grid with filename, `px | cm | DPI`, and KB under each thumbnail. The thumbnail can be chosen from a list or entered manually.

Diagnostics And Maintenance

Diagnostics contains `Formats`, `Information`, and `Doctor`. Maintenance actions are separate; global multithreading defaults live there, and `Clean I/O` clears managed `input\` and `output\` folders with confirmation.

Multithreading And RAM

Multithreading processes independent source images in parallel. It helps with batches, but large maps, TIFF/PDF rendering, and PNG output can occupy far more memory than the source file size on disk. Each extra worker can open another heavy image, so speedups can turn into paging or out-of-memory failures.

Practical guide:

- `16 GB RAM` - keep multithreading disabled; use `1-2` workers only for small files.
- `32 GB RAM` - usually `4` workers.
- `64 GB RAM` - `8-12` workers after checking peak memory.
- `128 GB+ RAM` - `16+` can be tested, but memory must be watched.

The global Maintenance setting is the default for modes without a local override. A local mode switch affects only the current run and does not change Workbench/I/O.

Main Folders

<code>input\</code>	source files
<code>output\</code>	processed results
<code>logs\</code>	operation logs
<code>report\</code>	GUI reports
<code>workspace\</code>	temporary work area
<code>config\</code>	manifest, GUI settings, themes, ICC profiles
<code>docs\</code>	documentation
<code>system_core\</code>	GUI/CLI code

CLI Examples

Use the project runtime:

```
runtime\python.exe system_core\main.py --help
runtime\python.exe system_core\main.py convert --help
runtime\python.exe system_core\main.py resize-screen --help
runtime\python.exe system_core\main.py images-to-pdf --help
```

Before Release

Run compile checks, `system_core\doctor.py`, and the GUI smoke test described in `RELEASE_GUIDE_RU.md` / `RELEASE_GUIDE_EN.md`.

Canonical Workbench labels

Workbench uses the same Audion Image Tools public vocabulary in every project. Its buttons always keep the same order and labels: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**.

Reset returns to project `input/output` and does not delete files; **Delete** clears the current **Source** and **Target** only after confirmation. The exact Russian labels are **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**. The Workbench variants **Destination**, **Clear**, **Цель**, and **Очистить** are not used.

Project Structure And Launcher Reference

Status note: this is an early planning document. The current working GUI entry point is `launcher_gui.cmd`; `launcher_project.cmd` and `launcher_project_ru.cmd` remain as CLI/TUI launchers. For the current operator-facing behavior, use `README_RU.md`, `docs/USER_GUIDE_RU.md`, and `AGENTS.md`.

Project positioning

Audion Image Tools is a portable Python image-processing toolkit focused on:

- broad **input format ingestion**

- strong **batch conversion**
- reliable **normalization**
- practical **JPG/PNG-centric processing**
- large **project launcher** workflow with future **FZF** integration

This project should treat most formats as **input sources**, while the most feature-complete processing pipeline targets:

- **JPG / JPEG**
- **PNG**

These two formats are the main working formats for the project.

Core product logic

Principle 1 — Convert broadly, work deeply on JPG/PNG

Most supported formats should be accepted as input and converted into stable working formats.

Primary working outputs:

- JPG
- PNG

Extended or special inputs:

- BMP
- GIF
- TIFF / TIF
- TGA
- WebP
- AVIF
- HEIC / HEIF
- SVG / SVGZ
- PSD
- WMF
- CR2 / CR3 / DNG

- IFF / XIF (best effort only)

Principle 2 — Do not touch `launcher_tools`

`launcher_tools` remains untouched.

All project-facing workflows live under a **large project launcher**:

- `launcher_project.cmd`
- `launcher_project.fzf`

Optional helper launchers may be added later if the menu becomes too large.

Principle 3 — FZF-first future

The launcher design should be prepared for:

- plain CMD menu now
- FZF-driven selection later
- same command architecture underneath

The FZF launcher should not redefine business logic.

It should only provide a better selection UI over the same workflow commands.

Recommended repository layout

```
Audion_Image_Tools/  
|  
├─ README.md  
├─ README.en.md  
├─ README.ru.md  
├─ LICENSE  
├─ requirements.txt  
├─ requirements-extended.txt  
├─ pyproject.toml  
|  
├─ launcher_project.cmd  
├─ launcher_project.fzf  
├─ launcher_project.ps1  
|  
├─ launcher_tools.cmd  
├─ launcher_tools.fzf  
|  
├─ install_environment.cmd  
├─ install_environment.ps1  
├─ verify_environment.cmd  
├─ verify_environment.ps1  
|  
├─ input/  
├─ output/  
├─ workspace/  
├─ logs/  
├─ presets/  
├─ fonts/  
├─ profiles/  
├─ examples/  
├─ docs/  
|  
├─ system_core/  
|   └─ cli/  
|       └─ main.py  
|       └─ commands_convert.py  
|       └─ commands_normalize.py  
|       └─ commands_resize.py  
|       └─ commands_canvas.py  
|       └─ commands_tiff.py
```

```
| | |─ commands_contact_sheet.py
| | |─ commands_watermark.py
| | |─ commands_batch.py
| | |─ commands_inspect.py
| |
| |─ pipeline/
| | |─ job_models.py
| | |─ pipeline_runner.py
| | |─ metadata_policy.py
| | |─ color_pipeline.py
| | |─ geometry_pipeline.py
| | |─ save_pipeline.py
| |
| |─ io/
| | |─ discover.py
| | |─ naming.py
| | |─ manifest.py
| | |─ sidecar.py
| | |─ reporting.py
| |
| |─ adapters/
| | |─ adapter_pillow.py
| | |─ adapter_heif.py
| | |─ adapter_svg.py
| | |─ adapter_raw.py
| | |─ adapter_psd.py
| | |─ adapter_fallback.py
| |
| |─ transforms/
| | |─ exif_fix.py
| | |─ convert.py
| | |─ grayscale.py
| | |─ alpha.py
| | |─ dpi.py
| | |─ resize.py
| | |─ crop.py
| | |─ aspect_ratio.py
| | |─ color_profile.py
| | |─ trim.py
| | |─ watermark.py
| | |─ contact_sheet.py
| | |─ animation.py
| |
```

```
|   ├── presets/
|   |   ├── jpeg_presets.py
|   |   ├── png_presets.py
|   |   ├── size_presets.py
|   |   ├── aspect_presets.py
|   |   └── color_presets.py
|   |
|   ├── inspect/
|   |   ├── image_info.py
|   |   ├── metadata_info.py
|   |   ├── icc_info.py
|   |   └── frame_info.py
|   |
|   ├── ui/
|   |   ├── menu_data.py
|   |   ├── menu_render.py
|   |   └── fzf_bridge.py
|   |
|   ├── utils/
|   |   ├── paths.py
|   |   ├── console.py
|   |   ├── validation.py
|   |   ├── progress.py
|   |   └── safe_ops.py
|   |
|   └── license/
|       ├── collect_third_party_licenses.py
|       └── Collect-ThirdPartyLicenses.ps1
|
|── tests/
|   ├── test_convert.py
|   ├── test_resize.py
|   ├── test_color_profile.py
|   ├── test_tiff_frames.py
|   └── test_naming.py
|
└── third_party_licenses/
```

Input format support tiers

Tier A — stable core

These should be first-class supported in v1:

- BMP
- JPG / JPEG
- PNG
- TGA
- TIFF / TIF
- GIF
- WebP
- AVIF

Tier B — extended adapters

These should be supported through optional dependencies:

- HEIC / HEIF
- SVG / SVGZ
- CR2 / CR3 / DNG

Tier C — best-effort / fallback

These may work but must not be overpromised:

- PSD
- WMF
- IFF
- XIF

Main processing groups

1. Convert

Purpose: broad input to stable working outputs.

Main modes

- Convert to JPG
- Convert to PNG
- Convert to JPG + PNG
- Batch convert mixed inputs to chosen target

JPG presets

- 60
- 75
- 90

PNG presets

Use compression presets rather than fake visual "quality":

- Fast
- Balanced
- Strong

2. Normalize / Fix Image

Purpose: make source images predictable before other work.

Included actions

- EXIF rotation fix
- safe mode conversion
- palette / RGBA / LA / CMYK handling
- embedded ICC detection
- convert profile to sRGB
- optional convert profile to CMYK
- strip broken metadata
- normalize output mode for target format

Safe conversions

Examples:

- RGBA -> JPG with white background
 - RGBA -> JPG with black background
 - RGBA -> JPG with custom background
 - CMYK -> sRGB JPG/PNG
 - P / indexed -> RGB
 - LA -> RGB or L
-

3. DPI / Resolution Metadata

Purpose: write or normalize DPI metadata without arbitrary geometric resampling.

Main modes

- keep original geometry, write new DPI
- set 72 / 96 / 150 / 300 / 600 DPI
- preserve original DPI if present
- strip DPI metadata

Important policy

This mode changes **metadata intent**, not actual pixel dimensions.

4. Screen Fit / Resize

Purpose: prepare wallpaper, screen, display, and presentation assets.

Target presets

- 1920x1080
- 2560x1440
- 3840x2160

Geometry modes

- fit and crop
 - contain
 - pad
 - fit by height
 - fit by width
-

5. Aspect Ratio Fit

Purpose: crop or frame to practical ratios.

Ratio presets

- 16:9 landscape
- 9:16 portrait
- A4 portrait
- A4 landscape
- A3 portrait
- A3 landscape

Modes

- crop to ratio
 - pad to ratio
 - fit to ratio without changing source geometry beyond required crop
 - keep original resolution where possible
-

6. Grayscale

Purpose: convert input images to grayscale working outputs.

Modes

- grayscale to JPG
- grayscale to PNG

- grayscale preserve original format where safe
- grayscale after normalization
- grayscale after crop/resize

This should be a first-class menu entry, not a hidden option.

7. TIFF Tools

Purpose: make multi-page TIFF practically usable.

Main modes

- extract all pages to numbered PNG
- extract first page only
- extract all pages to numbered JPG
- inspect TIFF frame/page count

Naming rule

Use numbered prefixes:

- 001_...
 - 002_...
 - ...
 - 999_...
-

8. Contact Sheet

Purpose: quick visual review of batches.

Modes

- contact sheet from folder
- contact sheet after conversion
- contact sheet after normalization
- fixed grid

- auto grid
 - filename labels on/off
-

9. Watermark / Text Overlay

Purpose: add reusable production marks.

Modes

- text watermark
 - image watermark
 - corner placement
 - centered placement
 - tiled watermark
 - opacity control
 - margin control
-

10. Border Trim

Purpose: remove white or near-uniform borders from scans and exports.

Modes

- white trim
 - black trim
 - auto edge trim
 - threshold controlled trim
-

11. Animation / Multi-frame Policy

Purpose: define how animated or multi-frame sources are handled.

Policies

- first frame only
- all frames
- flatten to single output
- export frames to sequence

Applies to:

- GIF
- animated WebP
- multi-page TIFF

Recommended launcher design

Main launcher

launcher_project.cmd

This is the large primary launcher.

It should contain grouped sections such as:

1. Convert
2. Normalize / Fix
3. DPI / Metadata
4. Screen Fit / Resize
5. Aspect Ratio Tools
6. JPG / PNG Advanced Tools
7. TIFF Tools
8. Contact Sheet
9. Watermark / Overlay
10. Border Trim
11. Inspect / Report
12. Batch Presets

13. Open Input Folder
14. Open Output Folder
15. Open Workspace
16. Cleanup Project Workspace

FZF launcher

`launcher_project.fzf`

This should mirror the same operations with searchable selection.

FZF should be ideal for large operation count.

It should support:

- fuzzy search
- grouped operation names
- short descriptions
- preset selection
- direct execution of matching action

Example naming style:

- `Convert -> Mixed Input -> JPG (Q75)`
- `Convert -> Mixed Input -> PNG (Balanced)`
- `Normalize -> EXIF + sRGB -> PNG`
- `Resize -> 2160p -> Crop`
- `Aspect -> A4 Portrait -> Crop`
- `TIFF -> Extract All Pages -> PNG`
- `JPG/PNG -> Grayscale`
- `Inspect -> Image Metadata Report`

Optional helper launchers

If the menu becomes too large, add optional helper launchers without replacing the main launcher:

- `launcher_convert.cmd`

- `launcher_normalize.cmd`
- `launcher_resize.cmd`
- `launcher_tiff.cmd`
- `launcher_batch.cmd`

These should remain secondary convenience launchers.

Main identity stays with:

- `launcher_project.cmd`
- `launcher_project.fzf`

Preset philosophy

Presets should be practical rather than abstract.

Recommended preset families

JPG

- Q60
- Q75
- Q90

PNG

- Fast
- Balanced
- Strong

DPI

- 72
- 96
- 150
- 300
- 600

Screen

- 1080p crop
- 1080p contain
- 1440p crop
- 1440p contain
- 2160p crop
- 2160p contain

Ratios

- 16x9 landscape
- 9x16 portrait
- A4 portrait
- A4 landscape
- A3 portrait
- A3 landscape

Color

- preserve if safe
- force sRGB
- force CMYK
- grayscale

Metadata policy

Recommended options:

- preserve all when safe
- preserve DPI + ICC only
- strip all metadata

This should be available globally or per job.

File naming policy

Output names should be stable and predictable.

Examples:

- `photo__jpg_q75.jpg`
- `photo__png_balanced.png`
- `photo__srgb.png`
- `photo__gray.jpg`
- `photo__2160p_crop.jpg`
- `scan__a4_portrait.png`
- `document__001.png`

Avoid random suffixes.

Batch workflow philosophy

A strong batch workflow matters more than single-file perfection.

Recommended batch flow:

1. detect files
 2. classify by adapter
 3. normalize if needed
 4. transform
 5. save
 6. report success / failure
 7. produce manifest
-

Suggested v1 command set

```
python -m system_core.cli.main convert --to jpg --quality 75
python -m system_core.cli.main convert --to png --preset balanced
python -m system_core.cli.main normalize --target-profile srgb
python -m system_core.cli.main normalize --target-profile cmyk
python -m system_core.cli.main dpi --set 300
python -m system_core.cli.main resize --preset 2160p --mode crop
python -m system_core.cli.main aspect --preset a4_portrait --mode crop
python -m system_core.cli.main gray
python -m system_core.cli.main tiff-extract --to png
python -m system_core.cli.main contact-sheet
python -m system_core.cli.main watermark --text "Audion"
python -m system_core.cli.main trim --mode white
python -m system_core.cli.main inspect
```

MVP priority order

Phase 1 — main production core

- mixed input discovery
- convert to JPG
- convert to PNG
- EXIF fix
- sRGB normalization
- alpha handling for JPG
- metadata policy
- grayscale
- resize to 1080p / 1440p / 2160p
- aspect ratio crop for 16:9 / A4 / A3

Phase 2 — practical power tools

- TIFF extraction
- contact sheet
- watermark / text overlay

- border trim
- inspect / manifest report

Phase 3 — extended adapters

- HEIC / HEIF
- SVG / SVGZ
- CR2 / CR3 / DNG
- fallback pipeline for PSD / WMF / niche inputs

Key design summary

The heart of the project should be:

- broad format ingestion
- strong conversion
- deep JPG/PNG workflows
- large project launcher
- clean future FZF transition
- `launcher_tools` left untouched

That is the most stable and scalable direction for **Audion Image Tools**.